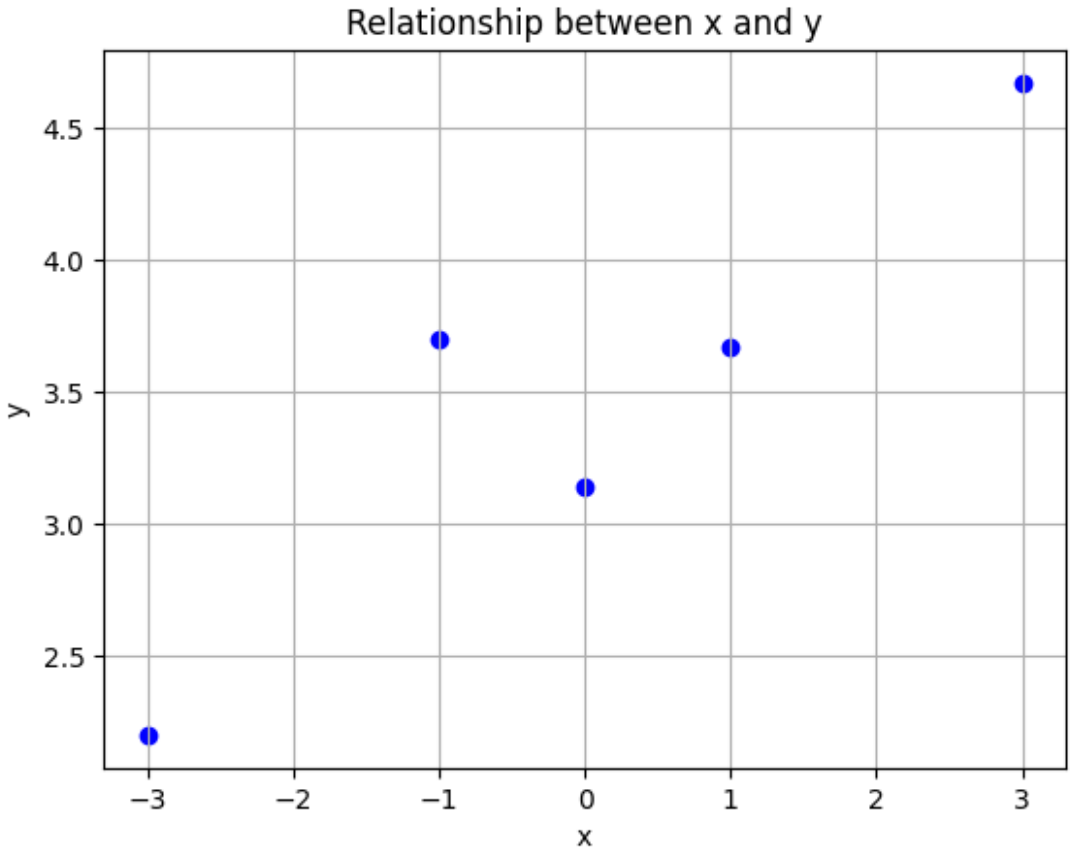
```python
# Step 1

import numpy as np
import matplotlib.pyplot as plt

X = np.array([-3, -1, 0.0, 1, 3]).reshape(-1, 1) # 5x1 vector, N=5,
D=1
y = np.array([2.2, 3.7, 3.14, 3.67, 4.67]).reshape(-1, 1) # 5x1 vector


plt.scatter(X, y, color='blue')
plt.title('Relationship between x and y')
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True)
plt.show()
```

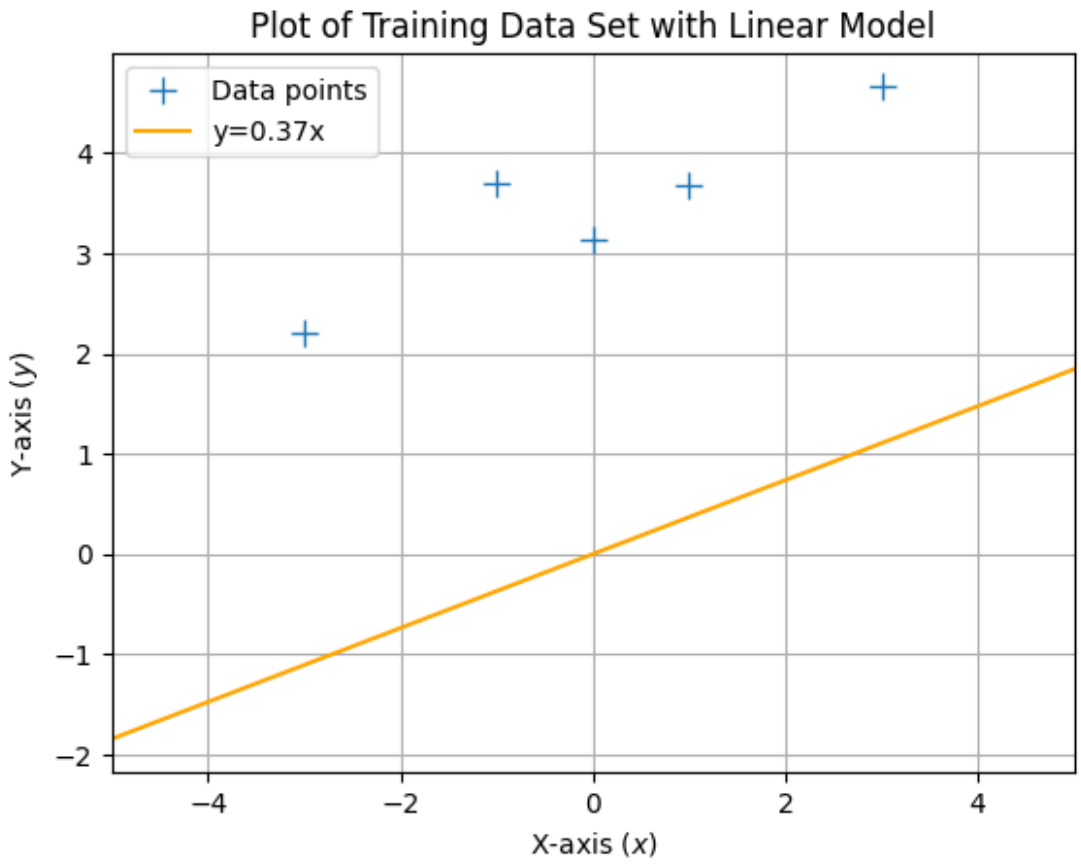



```python
# Step 2

import numpy as np

X = np.array([-3, -1, 0.0, 1, 3]).reshape(-1, 1)
y = np.array([2.2, 3.7, 3.14, 3.67, 4.67]).reshape(-1, 1)
```

```
def max_lik_estimate(X, y):
    inverse_term = np.linalg.inv(X.T @ X)  # Calculate the inverse of
X^T * X
    theta_ml = inverse_term @ (X.T @ y)    # Calculate θ_ML
    return theta_ml

theta_ml = max_lik_estimate(X, y)


print("Theta (θ_ML):", theta_ml)
```

```
Theta (θ_ML): [[0.369]]
```

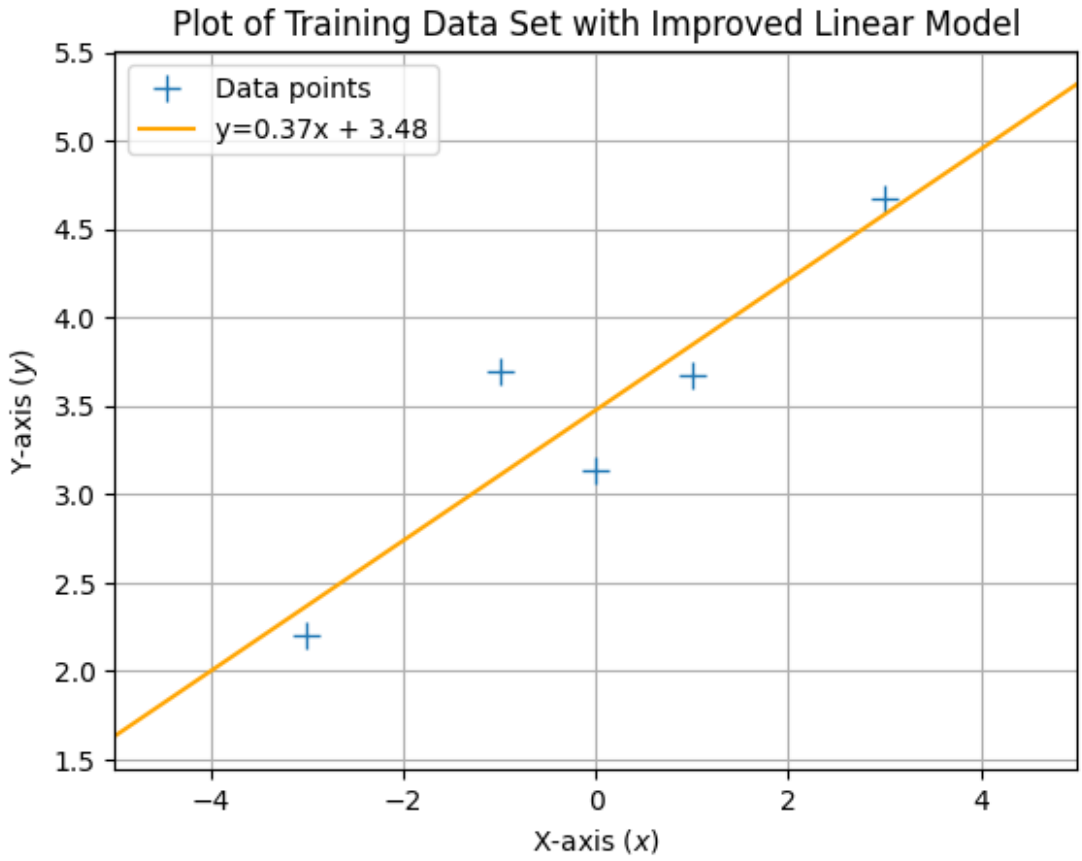
```
# Step 3

import numpy as np
import matplotlib.pyplot as plt


X = np.array([-3, -1, 0.0, 1, 3]).reshape(-1, 1)
y = np.array([2.2, 3.7, 3.14, 3.67, 4.67]).reshape(-1, 1)

def max_lik_estimate(X, y):
    inverse_term = np.linalg.inv(X.T @ X)
    theta_ml = inverse_term @ (X.T @ y)
    return theta_ml

theta_ml = max_lik_estimate(X, y)

Xtest = np.linspace(-5, 5, 100).reshape(-1, 1)
ml_prediction = Xtest @ theta_ml

plt.figure()
plt.plot(X, y, '+', markersize=10, label='Data points')
plt.plot(Xtest, ml_prediction, label=f'y={theta_ml[0][0]:.2f}x',
color='orange')
plt.xlabel("X-axis ($x$)")
plt.ylabel("Y-axis ($y$)")
plt.title("Plot of Training Data Set with Linear Model")
plt.xlim([-5, 5])
plt.legend()
plt.grid(True)
plt.show()
```




```
# Step 4

import numpy as np

X = np.array([-3, -1, 0.0, 1, 3]).reshape(-1, 1)
y = np.array([2.2, 3.7, 3.14, 3.67, 4.67]).reshape(-1, 1)

N, D = X.shape
X_aug = np.hstack([np.ones((N, 1)), X])

print("="*10)
print("X_aug = ")
print(X_aug)
print("="*10)

def max_lik_estimate(X, y):
    inverse_term = np.linalg.inv(X.T @ X)
    theta_ml = inverse_term @ (X.T @ y)
    return theta_ml

theta_ml_aug = max_lik_estimate(X_aug, y)

print("Theta (θ_ML) with bias term:", theta_ml_aug)
```

```
# In Step 4, the values differ from Step 2 because Step 2 only
calculates the slope of the line without an intercept.
#  This means the line must go through the origin (0,0), which may not
fit the data well.
# In Step 4, by adding a column of ones, the model can calculate both
the slope and the intercept.
# This allows the line to shift up or down, resulting in better
accuracy and a closer fit to the data points.

==========
X_aug =
[[ 1. -3.]
 [ 1. -1.]
 [ 1.  0.]
 [ 1.  1.]
 [ 1.  3.]]
==========
Theta (θ_ML) with bias term: [[3.476]
 [0.369]]

# Step 5

import numpy as np
import matplotlib.pyplot as plt

X = np.array([-3, -1, 0.0, 1, 3]).reshape(-1, 1)
y = np.array([2.2, 3.7, 3.14, 3.67, 4.67]).reshape(-1, 1)

N, D = X.shape
X_aug = np.hstack([np.ones((N, 1)), X])

def max_lik_estimate(X, y):
    inverse_term = np.linalg.inv(X.T @ X)
    theta_ml = inverse_term @ (X.T @ y)
    return theta_ml

theta_ml_aug = max_lik_estimate(X_aug, y)

Xtest = np.linspace(-5, 5, 100).reshape(-1, 1)
Xtest_aug = np.hstack([np.ones((Xtest.shape[0], 1)), Xtest])

ml_prediction_aug = Xtest_aug @ theta_ml_aug

plt.figure()
plt.plot(X, y, '+', markersize=10, label='Data points')
plt.plot(Xtest, ml_prediction_aug, label=f'y={theta_ml_aug[1][0]:.2f}x
+ {theta_ml_aug[0][0]:.2f}', color='orange')
plt.xlabel("X-axis ($x$)")
plt.ylabel("Y-axis ($y$)")
plt.title("Plot of Training Data Set with Improved Linear Model")
```

```
plt.xlim([-5, 5])
plt.legend()
plt.grid(True)
plt.show()
```




```
# Step 6

# Adding a column of ones to the input data allows the linear
regression model to include a bias term,
# or intercept, in its calculations.
# This means the model can adjust the line it fits to the data both up
and down,
# rather than being forced to go through the origin (0,0).
# This is important because real-world data often doesn't start at
zero,
# so having the intercept helps the model better capture the true
relationship between the input and output values.
# As a result, the model can make more accurate predictions and fit
the data points more closely.
```